Department of Computer Science and Engineering
University of Moratuwa

CS4533 – Concurrent Programming

Lab 04: Semaphores — Synchronization Problems

Module:	CS4533 – Concurrent Programming
Lab:	Lab 04 – Semaphores: Synchronization Problems
Duration:	2 hours (in-class)
Topics:	POSIX semaphores, signalling, capacity control, group coordination, barriers

Prerequisites

- Completed Labs 01–03 (threads, mutexes, condition variables, OpenMP)
- Understanding of mutual exclusion and thread signalling
- Compile all exercises with: gcc -pthread -O0 -o output source.c
- POSIX semaphores are in <semaphore.h> and link with -pthread

Learning Objectives

By the end of this lab, you should be able to:

- Use POSIX semaphores (sem_init, sem_wait, sem_post, sem_destroy) for thread synchronization
- Distinguish between binary semaphores (mutex-like) and counting semaphores (resource counting)
- Solve classic synchronization problems using semaphores
- Implement a reusable barrier primitive from scratch

Semaphore Quick Reference

A semaphore is an integer counter with two atomic operations: wait (decrement, blocks if zero) and post (increment, wakes a waiting thread). Unlike mutexes, semaphores can be posted by a thread that didn't do the wait — making them useful for signalling between threads.

```
#include <semaphore.h>

sem_t sem;

// Initialize: value = initial count
//   0 = shared between threads (not processes)
sem_init(&sem, 0, value);

// Wait (P operation): decrement, block if 0
sem_wait(&sem);
```

```
// Post (V operation): increment, wake one waiter
sem_post(&sem);

// Clean up
sem_destroy(&sem);
```

Key difference from mutexes: a semaphore initialized to N allows N threads to pass sem_wait before any blocks. This makes counting semaphores ideal for managing N resources or limiting concurrency.

Naming convention: Textbooks use different names for the same operations. sem_wait() is also called P(), down(), or wait(). sem_post() is also called V(), up(), or signal(). The POSIX names (sem_wait/sem_post) are what you use in code.

Exercise 1: Traffic Intersection

Problem

A narrow single-lane bridge connects two towns. Cars can travel in either direction (North-to-South or South-to-North), but the bridge only allows traffic in one direction at a time. Multiple cars travelling in the same direction may be on the bridge simultaneously (up to a maximum of 3 for safety). Cars from the opposite direction must wait until the bridge is clear.

Requirements

- Create car threads that arrive from random directions. Each car has an ID and a direction.
- A car entering the bridge prints: "Car <id> [N->S]: entering bridge"
- A car exiting the bridge prints: "Car <id> [N->S]: exiting bridge"
- Simulate crossing time with a short usleep() call.
- Use semaphores to enforce: (a) only one direction at a time, (b) at most 3 cars on the bridge simultaneously.
- Create 10 cars in each direction (20 total), arriving with random delays.
- At the end, print a summary: total cars passed, any violations detected.

Validation

Your program must self-validate: maintain shared counters for northbound and southbound cars currently on the bridge. After each entry/exit, check that both counters are never non-zero simultaneously, and neither exceeds 3. Print any violations.

Skeleton Code

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define MAX_ON_BRIDGE 3
#define CARS_PER_DIR 10

typedef struct {
    int id;
```

```
    int direction;  // 0 = N->S, 1 = S->N
} car_t;

// TODO: Declare your semaphores and shared state here

void *car_thread(void *arg) {
    car_t *car = (car_t *)arg;
    const char *dir = car->direction == 0
                    ? "N->S" : "S->N";

    usleep(rand() % 100000);  // random arrival

    // TODO: Enter bridge (acquire access)
    printf("Car %2d [%s]: entering bridge\n",
           car->id, dir);

    usleep(rand() % 50000);   // crossing time

    printf("Car %2d [%s]: exiting bridge\n",
           car->id, dir);
    // TODO: Exit bridge (release access)

    return NULL;
}

int main() {
    srand(42);
    // TODO: Initialize semaphores
    // TODO: Create car threads
    // TODO: Join all threads
    // TODO: Print summary
    // TODO: Destroy semaphores
    return 0;
}
```

Questions

Q1.1: How many semaphores did you use and what does each one represent? Could you solve this with fewer semaphores?

Q1.2: Is it possible for one direction to starve in your solution? If so, how would you fix it?

Exercise 2: Bridge Construction Zone

Problem

A road construction zone on a bridge has a narrow passage that allows vehicles in only one direction at a time. There are two types of vehicles: supply trucks heading TO the construction site, and empty trucks RETURNING from the site. The passage can hold up to N vehicles simultaneously, but only if they are all going in the same direction. When a vehicle from the opposite direction wants to enter, it must wait until the passage is completely empty.

Requirements

- The passage capacity N is provided as a command-line argument.
- Create S supply truck threads and R return truck threads (e.g., S=15, R=15).
- Each truck thread: arrives with a random delay, enters the passage, spends a random time inside (simulating travel), then exits.
- Use semaphores to enforce: (a) only one direction at a time, (b) at most N vehicles in the passage.
- Print timestamped log messages showing entry/exit and current vehicle counts:

```
[  0.012s] Supply  5: entered (supply=3, return=0)
[  0.025s] Supply  5: exited  (supply=2, return=0)
[  0.030s] Return  2: entered (supply=0, return=1)
```

- Self-validate: log and count any violations (mixed directions, capacity exceeded).
- At the end, print: total vehicles passed, direction violations, capacity violations.

Design Considerations

Think about how this differs from Exercise 1. The capacity is now variable (N from the command line), and you need to track counts for each direction to manage the direction switching logic. Consider using a combination of counting semaphores (for capacity) and binary semaphores (for direction control).

Questions

Q2.1: Explain your semaphore design: how many semaphores do you use, what are their initial values, and what does each represent?

Q2.2: Run with N=1, N=3, and N=5 (with S=15, R=15 each time). How does increasing N affect throughput (total time for all vehicles to pass)? Present timing results.

Q2.3: Compare this solution with the reader-writer problem you implemented in Lab 2 using mutex + condition variables. Which approach is cleaner? Which gives you more control over fairness?

Exercise 3: The Barbershop Problem

Problem

A barbershop has 1 barber, 1 barber chair, and M waiting chairs. The barber follows this routine: if there are no customers, the barber sleeps in the barber chair. When a customer arrives, if all waiting chairs are full, the customer leaves ("balks"). Otherwise, the customer sits in a waiting chair. If the barber is sleeping, the customer wakes the barber. The barber then serves customers one at a time until no one is left, then goes back to sleep.

Requirements

- The number of waiting chairs M is provided as a command-line argument.
- Create C customer threads (e.g., C=20) that arrive with random delays.
- The barber is a separate thread that loops: sleep → wake → serve all waiting → sleep again.
- Use semaphores to coordinate:
  - Customers waiting for the barber
  - The barber sleeping/waking
  - Access to the waiting chair count
  - Customers waiting for their haircut to finish

- Print log messages:

```
Customer  5: arrived (waiting: 3/5)
Customer 12: no seats, leaving
Barber: sleeping...
Barber: woke up, serving customer 5
Barber: finished customer 5
Customer  5: done, leaving happy
```

- At the end, print: total customers served, total customers turned away, and verify their sum equals C.

Questions

Q3.1: Run with M=1, M=3, and M=10 (C=20 each time). How does the number of waiting chairs affect the number of customers turned away? Present a table.

Q3.2: What happens if customers arrive very fast (reduce the arrival delay)? Does your solution still work correctly? Test it and report.

Q3.3: How would you extend this to support multiple barbers? Describe the semaphore changes needed (implementation not required).

Exercise 4: The River Crossing Problem

Problem

Hackers and Serfs arrive at a river bank to cross using a boat that holds exactly 4 people. For the boat to sail, the group of 4 must be one of the following valid combinations:

- 4 Hackers
- 4 Serfs
- 2 Hackers + 2 Serfs

Any other combination (such as 3+1) is NOT allowed. Threads must coordinate to form valid groups before anyone can board. Once a valid group of 4 is assembled, they board, one of them is designated the captain who calls row(), and all four cross together.

Requirements

- Create H hacker threads and S serf threads (e.g., H=12, S=12) arriving with random delays.
- Threads must wait at the river bank until a valid group of 4 can be formed.
- Use semaphores to coordinate group formation and boarding.
- The last person to complete a valid group signals the others to board.
- Print log messages showing group formation and crossing:

```
Hacker  3: arrived at bank
Serf    7: arrived at bank
Hacker  3: boarding (group: 2H + 2S)
Serf    7: boarding (group: 2H + 2S)
Hacker  3: rowing as captain!
Crossing complete: [H3, H5, S7, S2]
```

- At the end, print: total crossings, any invalid groups detected.

Hints

- Maintain counters for waiting hackers and waiting serfs, protected by a mutex (or binary semaphore).
- When a thread arrives, check if a valid group can be formed. If so, signal the group members.
- You may use a barrier or multiple semaphores to ensure all 4 board before the boat departs.
- Be careful: a thread that signals others must not proceed to form a new group until the current boat has departed.

Questions

Q4.1: Explain your semaphore design. How do you ensure only valid groups of 4 board? How do you prevent partial groups?

Q4.2: Is it possible for your solution to deadlock? If so, under what conditions? If not, explain why not.

Q4.3: Run with H=4, S=4 and then H=8, S=0. Does the second case work correctly? What about H=6, S=2 — can all threads cross? Explain.

Exercise 5: Reusable Barrier

Problem

Implement a reusable barrier that synchronizes N threads across multiple rounds. At each barrier point, all N threads must arrive before any thread proceeds to the next round. Unlike a one-shot barrier, your implementation must work correctly across multiple consecutive barrier calls.

Background

A naive barrier using a single counter and semaphore breaks when reused: threads from round 2 can arrive at the barrier before slow threads from round 1 have passed through. The classic solution uses a two-phase (turnstile) approach or alternating barriers.

Application: Iterative Averaging

Use your barrier in a practical scenario. N threads share a global array. In each iteration, every thread sets its element to the average of its neighbours. All threads must finish one iteration before any starts the next (otherwise a thread reads a neighbour's new value instead of the old one). Run for K iterations.

Serial Reference Code

```
#include <stdio.h>

#define SIZE 10
#define ITERATIONS 5

double arr[SIZE];
double temp[SIZE];

int main() {
    // Initialize
    for (int i = 0; i < SIZE; i++)
        arr[i] = (i == 0 || i == SIZE - 1)
                 ? 100.0 : 0.0;
```

```
    for (int iter = 0; iter < ITERATIONS; iter++) {
        // Average neighbours (skip boundaries)
        for (int i = 1; i < SIZE - 1; i++)
            temp[i] = (arr[i-1] + arr[i] + arr[i+1])
                    / 3.0;

        // Copy back
        for (int i = 1; i < SIZE - 1; i++)
            arr[i] = temp[i];

        printf("Iter %d: ", iter);
        for (int i = 0; i < SIZE; i++)
            printf("%.1f ", arr[i]);
        printf("\n");
    }
    return 0;
}
```

Your Task

- Implement a barrier_t struct and functions: barrier_init(), barrier_wait(), barrier_destroy() using only semaphores.
- Parallelize the iterative averaging: each thread handles a contiguous chunk of the array. After each iteration, all threads wait at the barrier before proceeding.
- Verify that the parallel version produces identical output to the serial version.
- Test with SIZE=100, ITERATIONS=20, and varying numbers of threads (2, 4, 8).

Questions

Q5.1: Why can't a simple counter + single semaphore work as a reusable barrier? Describe the specific failure scenario.

Q5.2: Does your parallel version produce the same results as the serial version? If there are small floating-point differences, explain why.

Submission Guidelines

- Submit source files (.c), one per exercise, named as: Lab04_<IndexNumber1>_<IndexNumber2>_Ex<N>.c
- Submit one PDF report containing: code output (terminal screenshots), log excerpts, validation summaries, timing tables, and answers to all questions.
- Compile all exercises with: gcc -pthread -O0 -o output source.c
- Submit via Moodle before the deadline.
